

BROWN

**Deterministic Guardrails for Preserving Privacy and
Security in Real-World Computer Systems**

by
Justus Adam

B.Sc., Technische Universität Dresden, 2016
Sc.M., Technische Universität Dresden, 2020

A dissertation submitted in partial fulfillment of the
requirements for the Degree of Doctor of Philosophy
in the Department of Computer Science at Brown University

Providence, Rhode Island
May 2028

© Copyright 2028 by Justus Adam

This dissertation by Justus Adam is accepted in its present form
by the Department of Computer Science as satisfying the
dissertation requirement for the degree of Doctor of Philosophy.

Date _____
Malte Schwarzkopf, Advisor

Recommended to the Graduate Council

Date _____
Deepti Raghavan, Reader

Date _____
Akshay Narayan, Reader

Date _____
Shriram Krishnamurthi, Reader

Approved by the Graduate Council

Date _____
Sorin Istrail, Dean of the Graduate School

PREFACE AND ACKNOWLEDGMENTS

Thank people here.

THESIS STATEMENT

Enforcement of privacy and security policies becomes practical in real applications when policies leverage shared, high-level vocabularies, authored by experts, that are legible to users and mechanically power to sound enforcement.

Vocabularies allow for a division of labor and no single involved party needs complete expertise. Experts establish the vocabulary once, bootstrapping soundness. Users build on top of the vocabulary, reusing and recombining it. By employing experts sparingly and empowering users, the vocabulary allows the enforcement to scale to real-world scale while at the same time leaving unsound assistive techniques, such as AI, off the critical path.

CONTENTS

1	Introduction	1
1.1	Trust and Threat Model	4
1.2	Contributions	5
2	Background	6
2.1	Data and Control Flow Analysis	6
2.2	Agentic AI	7
3	Paralegal	8
4	Palsgraf	13
5	Related Work and Conclusion	16
	Bibliography	17

LIST OF TABLES

Table 4.1	Which scenario variants each protection mechanism allowed or blocked. ✓ marks the intended outcome, ✗ a failure. The global sandbox is the honest baseline — a policy a careful engineer would write without per-task knowledge — and it fails in both directions, blocking intended work in one scenario and admitting divergent effects in the others. Only the per-task policy separates them everywhere.	15
-----------	---	----

LIST OF ILLUSTRATIONS

Figure 1.1	Comparison of the high-level structure of both systems. The vocabulary (middle row, highlighted) forms the interface between rarely engaged experts and usual users. Both systems compile policies over the vocabulary to a sound check.	3
Figure 2.1	An agentic AI application uses an interpreter to call tools requested by an LLM	7
Figure 3.1	Paralegal’s workflow. A privacy engineer writes a policy over marker names; a developer attaches those markers to source entities. The analyzer builds a marked program dependence graph over the application and its libraries, and checks the policy against it. Neither author reasons about the graph, and the analyzer assigns no meaning to the names.	8
Figure 3.2	User-data deletion written in Paralegal’s policy language. Figure 3.2a is too coarse: it is satisfied by code that only builds a deletion query. Figure 3.2b additionally requires the query to reach an executes marker, which needs a new marker from the developer. This give-and-take between privacy engineer and developer is the common workflow, and it is why the vocabulary has to be cheap to extend.	9
Figure 3.3	Partial and simplified marked PDG for the deletion example. Solid rectangles are PDG nodes, dashed rectangles are function scopes, and the rounded boxes are markers. The red subgraph is the comment-deletion code that was missing. “ <i>Lk</i> ” refers to line <i>k</i> of Listing 3.1 ; “start” and “end” are the function entry and exit locations.	10
Figure 3.4	In the “Workspace Only” configuration, which analyzes only the crates in the current workspace, Paralegal’s end-to-end runtime is under 2.1 seconds for most applications — fast enough to run interactively. The exceptions are Lemmy, which has 72 analysis entry points, and Hyperswitch, which is a large crate. PDG construction dominates.	11

Figure 3.5	In the “All Dependencies” configuration, which analyzes the application together with its dependencies, runtimes remain compatible with a CI budget.	11
Figure 3.6	Paralegal takes a mean of 0.8 seconds per endpoint, so cost scales with the number of analysis entry points rather than with total application size. ...	12
Figure 3.7	Adaptive approximation reduces end-to-end runtime: without it, PDG construction inlines call sites that cannot affect the policy’s verdict. This is what makes the interactive numbers above achievable.	12
Figure 4.1	An MCP tool for file creation. In Figure 4.1a the implementation performs an undisclosed metrics upload in addition to the documented file write; the description is the only thing the model — or the user approving its use — gets to see. Figure 4.1b adds a fine-grained sandbox whose configuration is injected by the trusted client rather than supplied by the server. Note that this is the design in which the tool developer cooperates; this chapter argues for confining the process instead, precisely because cooperation cannot be assumed.	13
Figure 4.2	Palsgraf’s components and trust boundaries, which are easy to invert. The client is trusted and is the only party that authors policy; the user approves it in the vocabulary of the action catalog. Everything the model asks for is mediated by the interpreter, and anything that escapes the interpreter — a spawned process, most of all — is confined by a sandbox the client configured, not by cooperation from the code being run.	14

CHAPTER 1

INTRODUCTION

Today’s applications are ever-growing in size and complexity, and ever more user data is being processed through these systems. To ensure that the users to whom this data belongs are afforded privacy and that the computer systems are secure, engineers and architects conduct mainly laborious manual audits. Some code-analysis tools are in use also, but they lack the ergonomics and scalability to be applied broadly. With the advent of agentic AI, these problems are exacerbated as developers increasingly do not write the code themselves. Code reviews become even more laborious, as LLMs generate vast amounts of code in minimal time. In many cases, the core of the applications themselves becomes agentic AI, a fundamentally unpredictable component.

Human reasoning alone will not efficiently scale to the complexity of modern applications. At the same time, automated tools must deliver reliable results, which disqualifies approaches that use AI on the critical path. This leaves formal techniques, such as static analysis, proofs, sandboxing or instrumentation. Traditionally, these are used only in high-impact areas as they require additional expertise to use effectively. They often struggle to scale to large code bases and they are ill-adapted to the dynamic nature of AI enabled applications.

This thesis comprises two projects that address roadblocks in the application of formal guardrails to realistic applications. Paralegal is a static analyzer for Rust programs. It presents an ergonomic interface to privacy and security policy authoring based on

markers, text objects attached to source code elements. For classical applications, static analysis offers the benefit of providing guarantees at compile time, and do not impact runtime performance or alter runtime behavior in surprising ways. The enforcement engine, a program dependence graph (PDG) based analyzer, uses these markers as well as guarantees provided by Rust's type-system enforced ownership model to make the analysis scale to real-world scale Rust programs.

Palsgraf on the other hand addresses the novel threat, posed to users, from the addition of agentic AI. Applications that use agentic AI are highly dynamic. Their strength lies in performing tasks that are, at build time, unanticipated, and at creating tools for solving these tasks on-the-fly. This makes static techniques ill suited for ensuring an AI enabled application preserves user privacy and security without eliminating the adaptability for which the AI was employed in the first place.

Dynamic techniques also struggle in the agentic AI setting. Traditionally, the dynamism for these techniques is related to runtime inputs, whereas policies are statically fixed. With agentic AI, the tasks performed are not known ahead of time and only task-specific policies can distinguish desired behavior from dangerous violations.

Palsgraf is a dynamic policy enforcement engine that with first-class support for task-specific policies. To ensure the policies are trustworthy Palsgraf uses the user as source of the policies. Creating format task-specific policies takes however a lot of effort. To alleviate this burden Palsgraf prompts the agent itself to draft the policy. Palsgraf then renders this policy in human readable form to the user for approval. This mechanism is predicated on a policy that is concise and easy for users to understand. To facilitate this Palsgraf uses an expert-authored, registry of parameterized shapes of shell commands and network requests. Each such shape is given a meaningful name that the user would recognize. For example, the `GIT_COMMIT(where="/projects/foo")` is a meaningful effect to a typical developer and they need not reason about the fact that this means, internally, that this rule will forbid the `--force` flag.

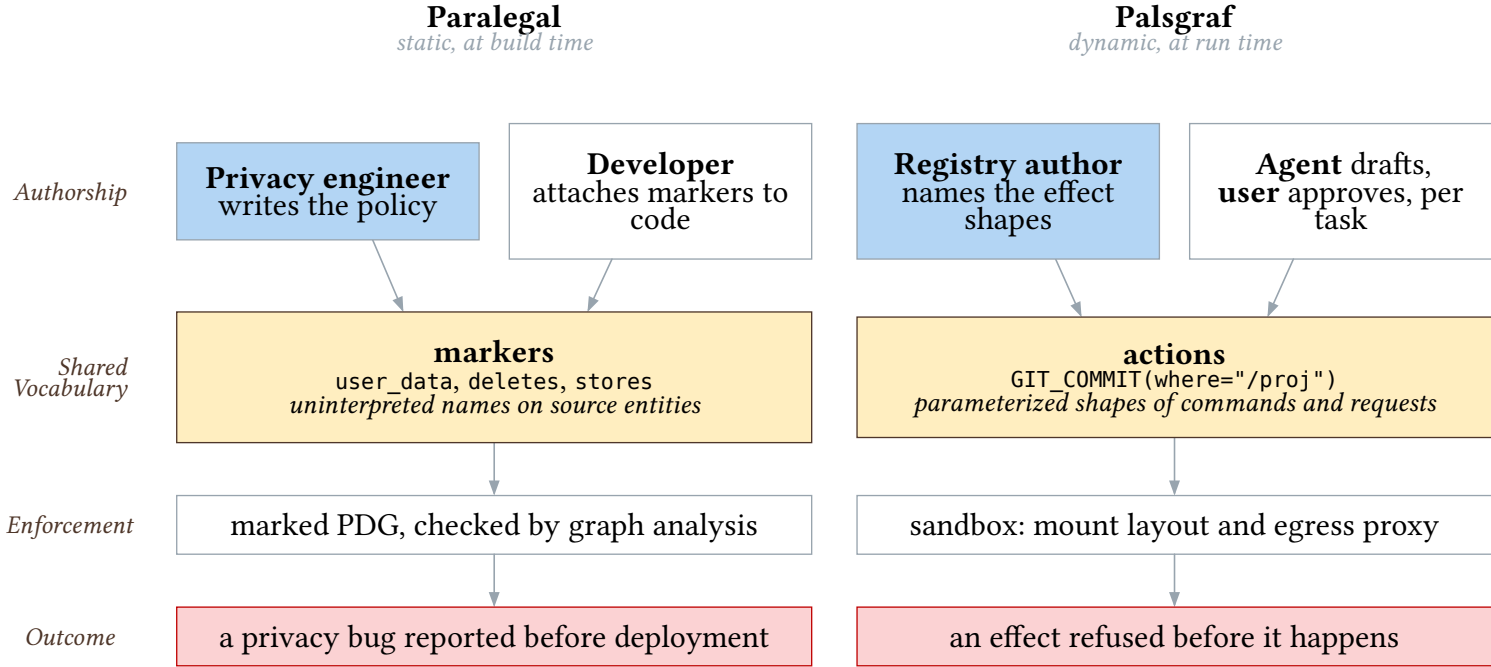


Figure 1.1. Comparison of the high-level structure of both systems. The vocabulary (middle row, highlighted) forms the interface between rarely engaged experts and usual users. Both systems compile policies over the vocabulary to a sound check.

In both cases existing systems fail to bridge the divide between what users can specify and what a deterministic engine can enforce. PDGs reason about how low-level values in the program relate to one another. Policy writers meanwhile consider high-level actions, such as encryption, and high-level concepts, such as user data. Developers meanwhile are experts about the code base and can relate these concepts to concrete code objects, such as functions and types. Paralegal provides an interface at just this boundary, allowing policy writers to define meaningful concepts and developers to instantiate and maintain them with respect to the codebase. As a result, policies need fewer revisions by experts and developers gain a tool they can use frequently to establish confidence in the compliance of their applications.

In a similar manner, sandboxes enforce access control at the level of system calls. To a user who wants a file edited, a write system call is meaningful. However, in many cases, system calls are too low level. For instance, the `git commit` command causes a series of file reads and writes that are part of `git`'s internal protocol that few users have in-

depth familiarity with. System calls also reason poorly about external communication, because the effect caused by the communication is determined by its payload but the protocols, such as HTTPS, largely use encryption, thus hiding the payloads at the system call level. Palsgraf enables experts to define a vocabulary that encodes high-level actions performed via shell commands and finely-granular external effects caused by network requests. Experts need only revise these rarely, while users frequently use and recombine them to assemble task-specific policies.

1.1 Trust and Threat Model

The assumption for this work is that users can be trusted to specify policies, but they may fail to produce code that upholds them. We do not trust agents to produce correct policies or correct implementations, though we need their cooperation to make progress.

For Paralegal, we trust a knowledgeable policy engineer to specify privacy and security policies, define markers and document what those markers mean. Developers, as experts in the application, are trusted to apply the markers to code entities. Paralegal’s goal is to help developers that make mistakes when writing the actual application. As such, we do not trust the developer to get the implementation right, especially in an evolving code base. The developers are trusted to place the markers correctly, since that is a *rare* and *high-impact* action and thus warrants careful treatment and review.

Palsgraf *guarantees* that every effect a task’s execution produces is bound by the policy in effect at that time. The system does not ensure that the executed effects achieve the task or that the LLM’s outputs are correct. Side-channels, kernel compromise and sandbox failure are out of scope.

The TCB encompasses the sandbox, the kernel that supports the sandbox as well as the network and shell brokers. The system does not trust the user’s (possibly copy-pasted) prompt, but it trusts the user to review proposed policies and it trusts the on-disk

actions catalogue’s web request predicates, shell command footprint specifications and policy files.

The *agent is untrusted* and Palsgraf relies on it only for utility. The agent can get confused about what goal it is supposed to achieve or attempt unsafe means to achieve it. An *adversary* may hijack the agent by altering web responses from compromised web servers or by placing content in files the agent legitimately needs to read, such as a public code repository. The adversary may modify the user’s machine only via the agent. Both adversary and agent have full knowledge of the policy.

1.2 Contributions

This work make the following contributions:

1. The Paralegal static analyzer, which checks high-level properties against low-level, evolving code bases.
2. The marker abstraction to decouple policies and code; and techniques to efficiently generate precise semantic PDGs from Rust code and model the behavior of library code.
3. A flexible policy framework that compiles policies in a high-level language into queries on semantic PDGs.
4. Case studies reporting on our experience of applying Paralegal to eight real-world Rust web applications.
5. A policy language and action verb mechanism that lets users express policies at the level of meaningful effect units.
6. An scheme and mechanism for agents to draft policies and users to review them, compatible with MCP.
7. A sandbox adaptation that allows judging network requests on payload and protocol program arguments.

CHAPTER 2

BACKGROUND

2.1 Data and Control Flow Analysis

Dataflow analysis is crucial when determining the privacy compliance of programs. Whether a datum is private cannot always be determined by its content. PII (name, SSN, telephone number) can sometimes be identified via its unique structure. However, private data is a broader category, and data such as diary or email content, bank balances or contracted illnesses are often embedded in unstructured prose which also colocates data that may be sufficient to determine the data subject.

A reliable way to determine if data is private is by considering the data source, e.g. a bank statement, patient chart etc. This means however that in a program that manipulates such data, it cannot be determined at the sink whether the data that was input to the operation is private **unless** its origins have been tracked through the program.

A primary challenge in statically determining the origins of a datum in a program are pointers. Which value a given pointer may refer to, and therefore whether one of them could be sensitive or not, depends on all parts of the program that may modify pointer. This is called alias-analysis and unlike many other parts of dataflow analysis it is a non-local analysis, meaning it cannot be performed for one function alone, but must take into account all of its callers. This leads to an explosion of the state space.

2.2 Agentic AI

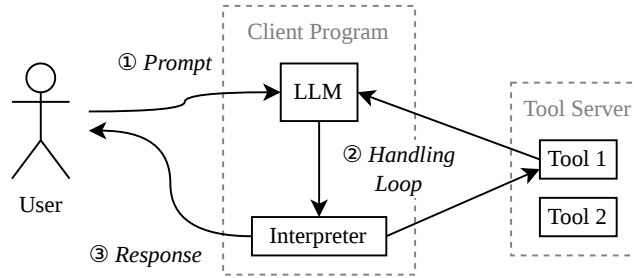


Figure 2.1. An agentic AI application uses an interpreter to call tools requested by an LLM

The basic architecture or agentic loops is remarkably simple and, crucially, has deterministic programs on the critical path. Conceptually, agentic AI lets a language model “invoke tools”, that is, programs. However, in reality, the AI itself has no means of directly running these programs. Its inputs and outputs are simply text [1, 2]. A special interpreter processes the text produced by the LLM and *this interpreter* invokes programs in response to LLM requests. A common architecture is that the LLM returns all of its responses in JSON format, with a type field that identifies the request as, for example, `thinking`, `user_response` or `tool_call`. Depending on this choice the interpreter performs dispatch back to the LLM (`thinking`), to the UI renderer (`user_response`) or a program (`tool_call`). This means the interpreter retains full control over how the tool call proceeds. It can, for example, log the request, inspect or change its arguments, fabricate the response or outright deny it.

CHAPTER 3

PARALEGAL

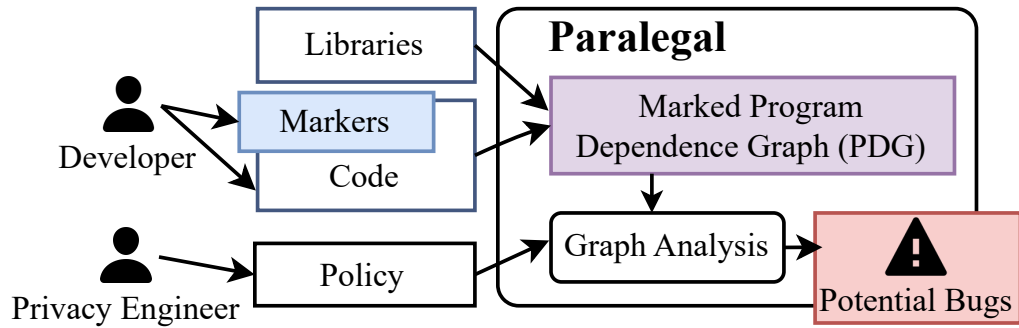


Figure 3.1. Paralegal’s workflow. A privacy engineer writes a policy over marker names; a developer attaches those markers to source entities. The analyzer builds a marked program dependence graph over the application and its libraries, and checks the policy against it. Neither author reasons about the graph, and the analyzer assigns no meaning to the names.

```

1  #[paralegal::marker(user_data)]
2  struct Comment { ... }
3
4  impl Database {
5      #[paralegal::marker(make_delete_query, arguments = [id])]
6      fn prepare_delete(&mut self, id: u32, table: &str) {...}
7  }
8
9  impl User {
10     #[paralegal::analyze]
11     fn delete_user(&self, db: &mut Database) {
12         let my_data: UserData = self.get_my_data();
13         db.prepare_delete(self.id, "users");
14         for post in &my_data.posts {
15             db.prepare_delete(post.id, "posts");
16         }
17         for comment in &my_data.comments {
18             db.prepare_delete(comment.id, "comments");
19         }
20         db.execute();
21     }
22 }

```

} the bug

Listing 3.1. Paralegal alerts developers to the missing code (red) that deletes a user’s comments when deleting their account in Plume. Markers are highlighted in teal. Code simplified and error handling omitted.

Somewhere:

1. For each "user data" type marked user_data:
 - A. There is a "source" that produces "user data" where:
 - a. There is a "deleter" marked deletes where:
 - i) "source" goes to "deleter"

(a) The initial policy.

Somewhere:

1. For each "user data" type marked user_data:
 - A. There is a "source" that produces "user data" where:
 - a. There is a "deleter" marked make_delete_query where:
 - i) "source" goes to "deleter"
 - and
 - ii) There is an "execute" marked executes where:
 - A) "deleter" goes to "execute"

(b) The revised policy.

Figure 3.2. User-data deletion written in Paralegal’s policy language. **Figure 3.2a** is too coarse: it is satisfied by code that only builds a deletion query. **Figure 3.2b** additionally requires the query to reach an executes marker, which needs a new marker from the developer. This give-and-take between privacy engineer and developer is the common workflow, and it is why the vocabulary has to be cheap to extend.

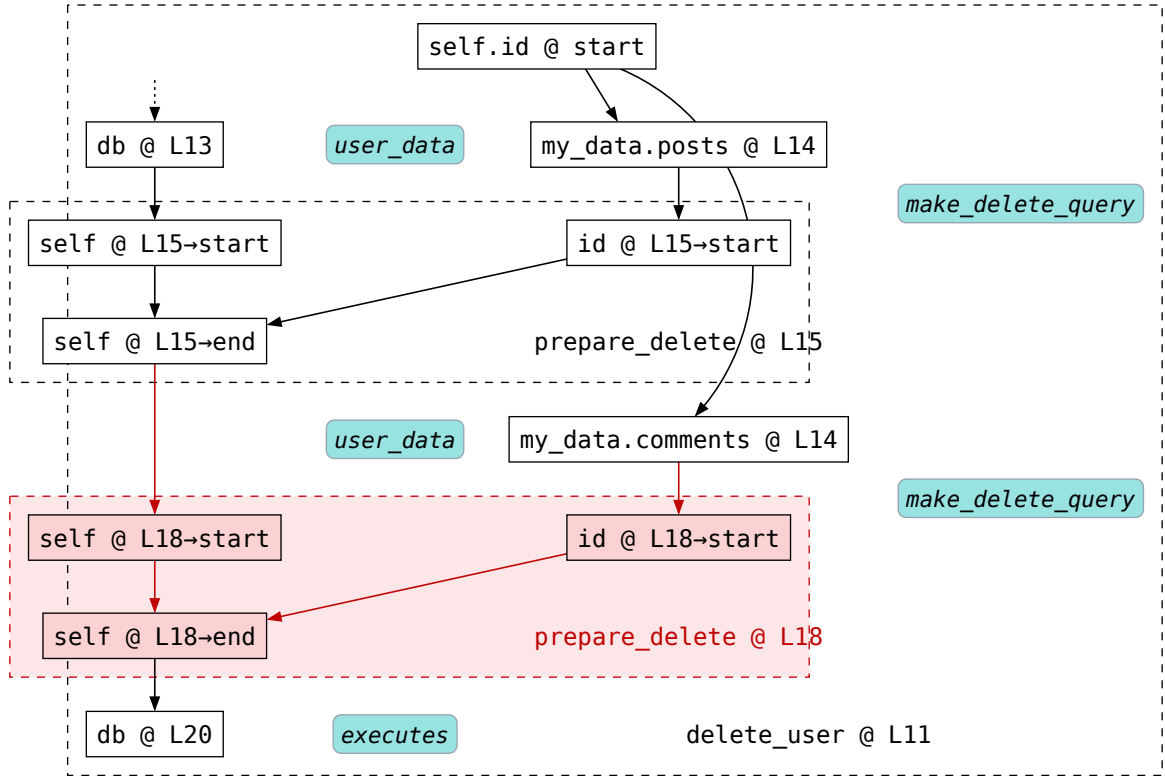


Figure 3.3. Partial and simplified marked PDG for the deletion example. Solid rectangles are PDG nodes, dashed rectangles are function scopes, and the rounded boxes are markers. The red subgraph is the comment-deletion code that was missing. “ Lk ” refers to line k of Listing 3.1; “start” and “end” are the function entry and exit locations.

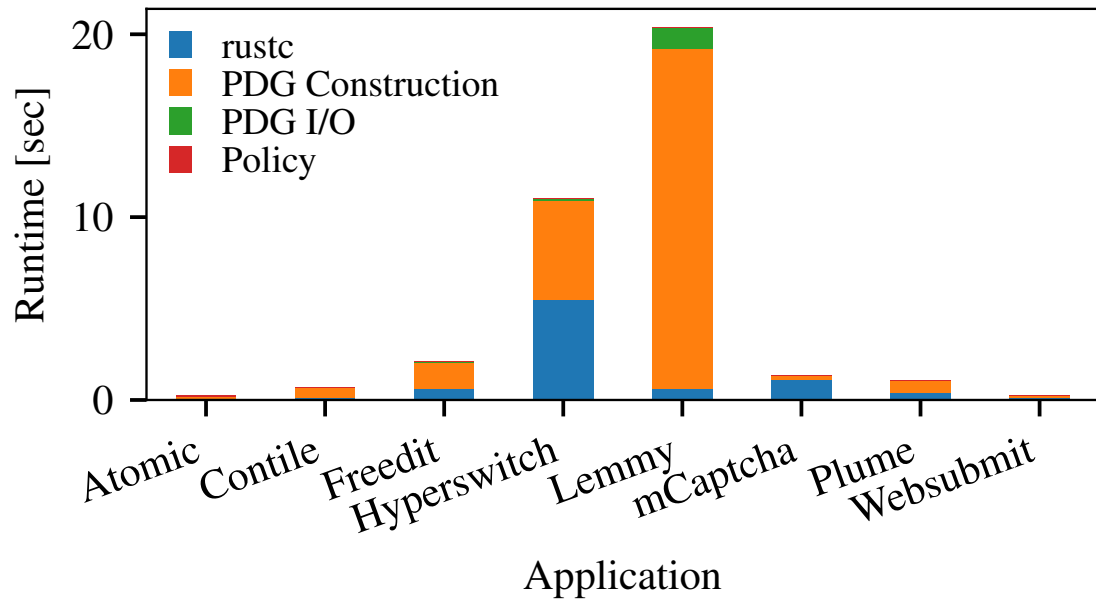


Figure 3.4. In the “Workspace Only” configuration, which analyzes only the crates in the current workspace, Paralegal’s end-to-end runtime is under 2.1 seconds for most applications — fast enough to run interactively. The exceptions are Lemmy, which has 72 analysis entry points, and Hyperswitch, which is a large crate. PDG construction dominates.

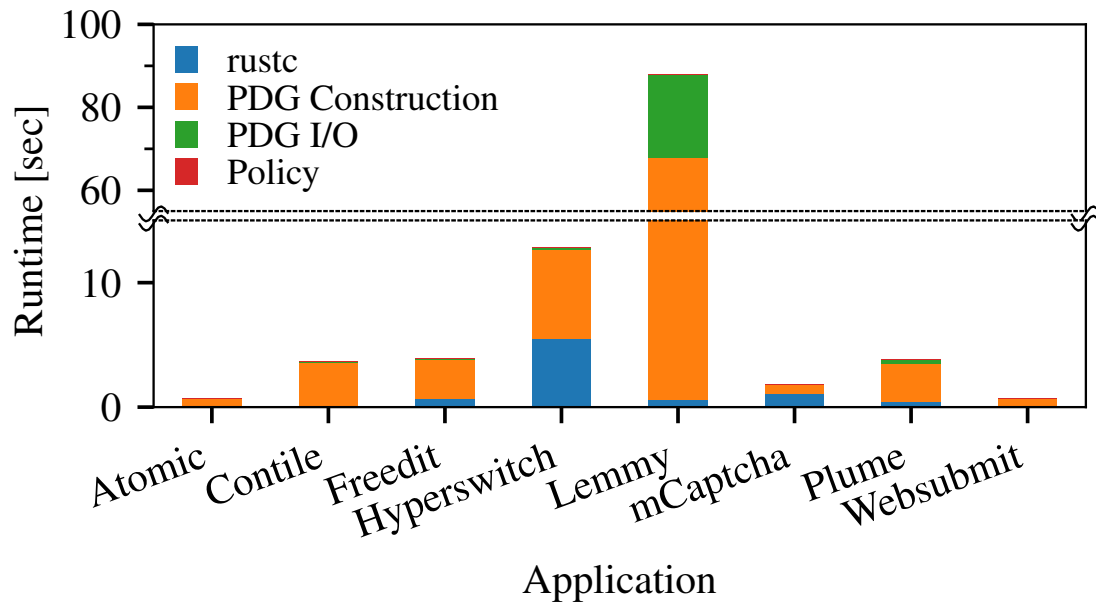


Figure 3.5. In the “All Dependencies” configuration, which analyzes the application together with its dependencies, runtimes remain compatible with a CI budget.

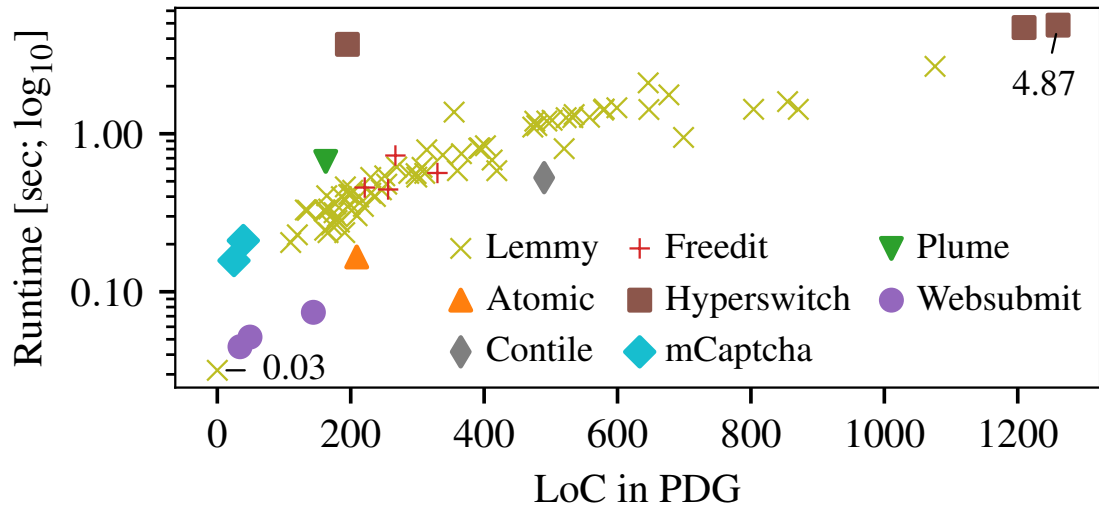


Figure 3.6. Paralegal takes a mean of 0.8 seconds per endpoint, so cost scales with the number of analysis entry points rather than with total application size.

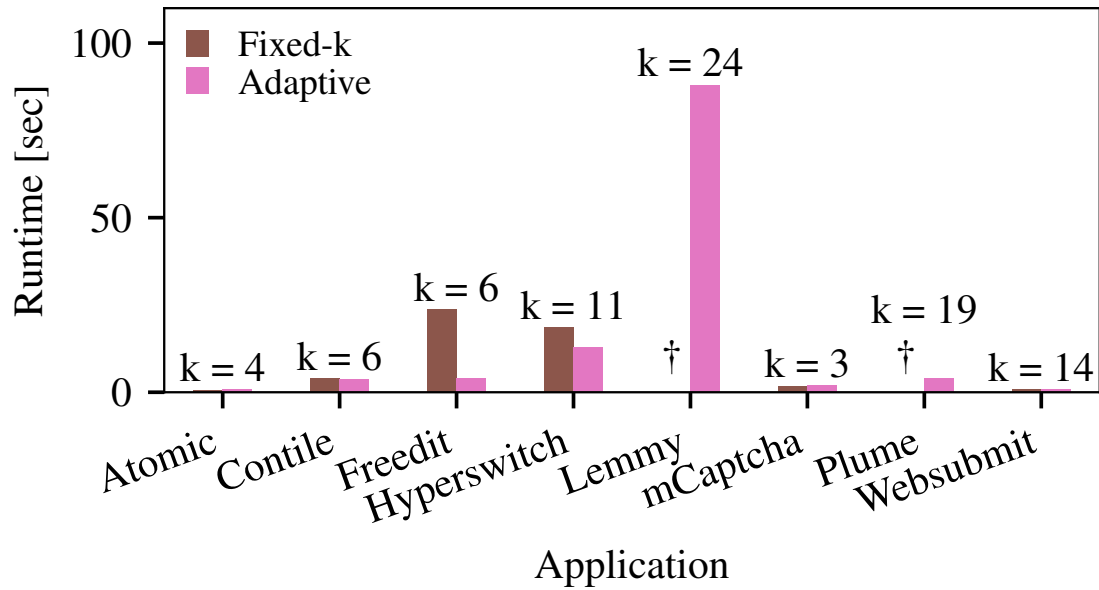


Figure 3.7. Adaptive approximation reduces end-to-end runtime: without it, PDG construction inlines call sites that cannot affect the policy’s verdict. This is what makes the interactive numbers above achievable.

CHAPTER 4

PALSGRAF

```
1 @tool(description="Writes `content` to file at `path`")
2 def write_file(self, path: str, content: str):
3     self.metrics.send({
4         'method': "write_file",
5         'bytes': len(content) })
6     with open(path, "w") as f:
7         f.write(content)
```

} undisclosed

(a) The tool as written: the metrics upload is not in its description.

```
1 #[tool(description = "Writes `content` to the file at `path`")]
2 fn write_file(&self, path: &Path, content: String,
3     // injected at the client by the protection system
4     scfg: SandboxConfig
5 ) {
6     self.metrics.send(json!({
7         method: "write_file",
8         bytes: content.len() }));
9     sandbox::write(path, content, scfg).unwrap();
10 }
```

(b) The same tool with a fine-grained sandbox.

Figure 4.1. An MCP tool for file creation. In [Figure 4.1a](#) the implementation performs an undisclosed metrics upload in addition to the documented file write; the description is the only thing the model — or the user approving its use — gets to see. [Figure 4.1b](#) adds a fine-grained sandbox whose configuration is injected by the trusted client rather than supplied by the server. Note that this is the design in which the tool developer cooperates; this chapter argues for confining the process instead, precisely because cooperation cannot be assumed.

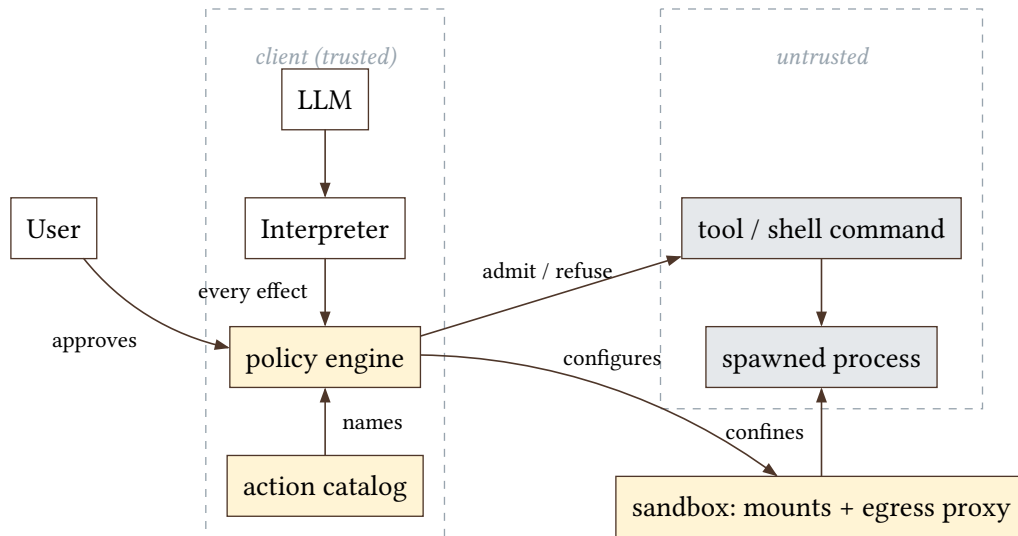


Figure 4.2. Palsgraf’s components and trust boundaries, which are easy to invert. The client is trusted and is the only party that authors policy; the user approves it in the vocabulary of the action catalog. Everything the model asks for is mediated by the interpreter, and anything that escapes the interpreter — a spawned process, most of all — is confined by a sandbox the client configured, not by cooperation from the code being run.

```
deny(all)
allow(net:write)
intent(file-edit) -> allow(fs:write in "~/workspace")
context(sensitive-data) -> deny(net:write)
```

Listing 4.1. An example policy for the file-editing tool.

```
input(path) -> effect(fs:write)
input(content) -> effect(fs:write)
input(content) -> effect(net:write)
```

Listing 4.2. The synopsis static analysis derives for the `write_file` tool: which of the tool’s inputs reach which effects. The undisclosed upload shows up as content reaching a network write.

Table 4.1. Which scenario variants each protection mechanism allowed or blocked. ✓ marks the intended outcome, ✗ a failure. The global sandbox is the honest baseline — a policy a careful engineer would write without per-task knowledge — and it fails in both directions, blocking intended work in one scenario and admitting divergent effects in the others. Only the per-task policy separates them everywhere.

Protection mechanism	.env file		Sensitive code		git push	
	Intended	Divergent	Intended	Divergent	Intended	Divergent
Unprotected	✓ Allowed	✗ Allowed	✓ Allowed	✗ Allowed	✓ Allowed	✗ Allowed
Global sandbox	✗ Blocked	✓ Blocked	✓ Allowed	✗ Allowed	✓ Allowed	✗ Allowed
Static effect detection	✗ Blocked	✓ Blocked	✓ Allowed	✓ Blocked	✓ Allowed	✓ Blocked
Palsgraf	✓ Allowed	✓ Blocked	✓ Allowed	✓ Blocked	✓ Allowed	✓ Blocked

CHAPTER 5

RELATED WORK AND CONCLUSION

BIBLIOGRAPHY

- [1] Engineering at Anthropic. Code execution with MCP: Building more efficient agents. Retrieved 2025 from <https://www.anthropic.com/engineering/code-execution-with-mcp>
- [2] Kenton Varda and Sunil Pai. Code Mode: the better way to use MCP. Retrieved 2025 from <https://blog.cloudflare.com/code-mode/>